

Firewall

The Linux kernel includes the [netfilter](#) subsystem, which is used to manipulate or decide the fate of network traffic headed into or through your server. All modern Linux firewall solutions use this system for packet filtering.

The kernel's packet filtering system would be of little use to administrators without a userspace interface to manage it. This is the purpose of the **iptables** utility: when a packet reaches your server, it will be handed off to the netfilter subsystem for acceptance, manipulation, or rejection based on the rules supplied to it from the userspace (via `iptables`). Thus, `iptables` is all you need to manage your firewall, if you're familiar with it, but many [frontends](#) are available to simplify the task. We'll take a look at the default frontend used in Ubuntu here.

ufw - Uncomplicated Firewall

The default firewall configuration tool for Ubuntu is `ufw`. Developed to ease `iptables` firewall configuration, `ufw` provides a user-friendly way to create an IPv4 or IPv6 host-based firewall.

`ufw` by default is initially disabled. From the [ufw\(8\)](#) manual page:

`ufw` is not intended to provide complete firewall functionality via its command interface, but instead provides an easy way to add or remove simple rules. It is currently mainly used for host-based firewalls.

Enable or disable ufw

To enable `ufw`, run the following command from a terminal prompt:

```
$ sudo ufw enable
```

To disable it, you can use the following command:

```
$ sudo ufw disable
```

Open or close a port

To open a port (SSH in this case):

Skip to content

Similarly, to close an opened port:

```
$ sudo ufw deny 22
```

Add or remove a rule

Rules can also be added using a **numbered** format:

```
$ sudo ufw insert 1 allow 80
```

To view the numbered format:

```
$ sudo ufw status numbered
```

To remove a rule, use `delete` followed by the rule:

```
$ sudo ufw delete deny 22
```

Allow access from specific hosts

It is possible to allow access from specific hosts or networks to a port. The following example allows SSH access from host `192.168.0.2` to any IP address on this host:

```
$ sudo ufw allow proto tcp from 192.168.0.2 to any port 22
```

Replace `192.168.0.2` with `192.168.0.0/24` to allow SSH access from the entire subnet.

The `--dry-run` option

Adding the `--dry-run` option to a `ufw` command will output the resulting rules, but not apply them. For example, the following is what would be applied if opening the HTTP port:

```
$ sudo ufw --dry-run allow http
```

```
*filter
:ufw-user-input - [0:0]
:ufw-user-output - [0:0]
:ufw-user-forward - [0:0]
:ufw-user-limit - [0:0]
:ufw-user-limit-accept - [0:0]
### RULES ###

### tuple ### allow tcp 80 0.0.0.0/0 any 0.0.0.0/0
-A ufw-user-input -p tcp --dport 80 -j ACCEPT

### END RULES ###
-A ufw-user-input -j RETURN
-A ufw-user-output -j RETURN
-A ufw-user-forward -j RETURN
-A ufw-user-limit -m limit --limit 3/minute -j LOG --log-prefix "[UFW LIMIT]: "
-A ufw-user-limit -j REJECT
-A ufw-user-limit-accept -j ACCEPT
COMMIT
Rules updated
```

Check the status

To see the firewall status, enter:

```
$ sudo ufw status
```

And for more verbose status information use:

```
$ sudo ufw status verbose
```

Note

If the port you want to open or close is defined in `/etc/services`, you can use the port name instead of the number. In the above examples, replace `22` with `ssh`.

This is a quick introduction to using `ufw`. Please refer to the [ufw\(8\)](#) manual page for more information.

ufw application integration

Applications that open ports can include a `ufw` profile, which details the ports needed for the application to function properly. The profiles are kept in `/etc/ufw/applications.d`, and can be edited if the default ports have been changed.

Skip to content Applications have installed a profile, run the following in a terminal:



```
$ sudo ufw app list
```

Similar to allowing traffic to a port, using an application profile is accomplished by entering:

```
$ sudo ufw allow Samba
```

An extended syntax is available as well:

```
$ ufw allow from 192.168.0.0/24 to any app Samba
```

Replace Samba and 192.168.0.0/24 with the application profile you are using and the IP range for your network.

Note

There is no need to specify the **protocol** for the application, because that information is detailed in the profile. Also, note that the app name replaces the port number.

To view details about which ports and protocols, and so on, are defined for an application, enter:

```
$ sudo ufw app info Samba
```

Not all applications that require opening a network port come with ufw profiles, but if you have profiled an application and want the file to be included with the package, please file a bug against the package in Launchpad.

```
$ ubuntu-bug nameofpackage
```

IP masquerading

The purpose of IP masquerading is to allow machines with private, non-routable IP addresses on your network to access the Internet through the machine doing the masquerading. Traffic from your private network destined for the Internet must be manipulated for replies to be routable back to the machine that made the request.

To do this, the kernel must modify the **source** IP address of each packet so that replies will be routed back to it, rather than to the private IP address that made the request, which is impossible over the Internet. Linux uses **Connection Tracking** ([conntrack\(8\)](#)) to keep track of which connections belong to which machines and reroute each return packet accordingly. Traffic leaving your private network is thus “masqueraded” as having originated from your Ubuntu gateway machine. This process is referred to in Microsoft documentation as “Internet Connection Sharing”.

IP masquerading with ufw

IP masquerading can be achieved using custom ufw rules. This is possible because the current back-end for ufw is iptables-restore with the rules files located in /etc/ufw/*.rules. These files are a great place to add legacy iptables rules used without ufw, and rules that are more network gateway or bridge related.

The rules are split into two different files; rules that should be executed before ufw command line rules, and rules that are executed after ufw command line rules.

Enable packet forwarding

First, packet forwarding needs to be enabled in ufw. Two configuration files will need to be adjusted, so first, in /etc/default/ufw change the DEFAULT_FORWARD_POLICY to "ACCEPT":

```
DEFAULT_FORWARD_POLICY="ACCEPT"
```

Then, edit /etc/ufw/sysctl.conf and uncomment:

```
net/ipv4/ip_forward=1
```

Similarly, for IPv6 forwarding, uncomment:

```
net/ipv6/conf/default/forwarding=1
```

Add the configuration

Now add rules to the /etc/ufw/before.rules file. The default rules only configure the **filter** table, and to enable masquerading the nat table will need to be configured. Add the following to the top of the file, just after the header comments:

```
# nat Table rules
*nat
:POSTROUTING ACCEPT [0:0]

# Forward traffic from eth1 through eth0.
-A POSTROUTING -s 192.168.0.0/24 -o eth0 -j MASQUERADE

# don't delete the 'COMMIT' line or these nat table rules won't be processed
COMMIT
```

Skip to content ↗ not strictly necessary, but it is considered good practice to document changes, when modifying any of the rules files in /etc/ufw, make sure they are the last line for each table modified:

```
# don't delete the 'COMMIT' line or these rules won't be processed
COMMIT
```

For each **Table**, a corresponding `COMMIT` statement is required. In these examples only the `nat` and `filter` tables are shown, but you can also add rules for the `raw` and `mangle` tables.

Note

In the above example, replace `eth0`, `eth1`, and `192.168.0.0/24` with the appropriate interfaces and IP range for your network.

Restart `ufw`

Finally, disable and re-enable `ufw` to apply the changes:

```
$ sudo ufw disable && sudo ufw enable
```

IP masquerading should now be enabled. You can also add any additional `FORWARD` rules to the `/etc/ufw/before.rules`. It is recommended that these additional rules be added to the `ufw-before-forward` chain.

IP masquerading with iptables

`iptables` can also be used to enable masquerading.

Similarly to `ufw`, the first step is to enable IPv4 packet forwarding by editing `/etc/sysctl.conf` and uncomment the following line:

```
net.ipv4.ip_forward=1
```

If you wish to enable IPv6 forwarding also uncomment:

```
net.ipv6.conf.default.forwarding=1
```

Next, run the `sysctl` command to enable the new settings in the configuration file:

```
$ sudo sysctl -p
```

IP masquerading can now be accomplished with a single `iptables` rule, which may differ slightly based on your network configuration:

```
$ sudo iptables -t nat -A POSTROUTING -s 192.168.0.0/16 -o ppp0 -j MASQUERADE
```

[Skip to content](#)



The above command assumes that your private address space is `192.168.0.0/16` and that your Internet-facing device is `ppp0`. The syntax is broken down as follows:

- `-t nat` – the rule is to go into the NAT table
- `-A POSTROUTING` – the rule is to be appended (`-A`) to the `POSTROUTING` chain
- `-s 192.168.0.0/16` – the rule applies to traffic originating from the specified address space
- `-o ppp0` – the rule applies to traffic scheduled to be routed through the specified network device
- `-j MASQUERADE` – traffic matching this rule is to “jump” (`-j`) to the `MASQUERADE` target to be manipulated as described above

Also, each chain in the filter table (the default table, and where most – or all – packet filtering occurs) has a default **policy** of `ACCEPT`, but if you are creating a firewall in addition to a gateway device, you may have set the policies to `DROP` or `REJECT`, in which case your masqueraded traffic needs to be allowed through the `FORWARD` chain for the above rule to work:

```
$ sudo iptables -A FORWARD -s 192.168.0.0/16 -o ppp0 -j ACCEPT
$ sudo iptables -A FORWARD -d 192.168.0.0/16 -m state \
> --state ESTABLISHED,RELATED -i ppp0 -j ACCEPT
```

The above commands will allow all connections from your local network to the Internet and all traffic related to those connections to return to the machine that initiated them.

If you want masquerading to be enabled on reboot, which you probably do, edit `/etc/rc.local` and add any commands used above. For example add the first command with no filtering:

```
$ iptables -t nat -A POSTROUTING -s 192.168.0.0/16 -o ppp0 -j MASQUERADE
```

Logs

Firewall logs are essential for recognizing attacks, troubleshooting your firewall rules, and noticing unusual activity on your network. You must include logging rules in your firewall for them to be generated, though, and logging rules must come before any applicable terminating rule (a rule with a target that decides the fate of the packet, such as `ACCEPT`, `DROP`, or `REJECT`).

If you are using `ufw`, you can turn on logging by entering the following in a terminal:

```
$ sudo ufw logging on
```

To turn logging off in `ufw`, replace `on` with `off` in the above command:

[Skip to content](#) [iptables](#) instead of `ufw`, run:

```
$ sudo iptables -A INPUT -m state --state NEW -p tcp --dport 80 \
> -j LOG --log-prefix "NEW_HTTP_CONN: "
```

A request on port 80 from the local machine, then, would generate a log in [dmesg](#) that looks like this (single line split into 3 to fit this document):

```
[4304885.870000] NEW_HTTP_CONN: IN=lo OUT= MAC=00:00:00:00:00:00:00:00:00:00:00:00 SRC=127.0.0.1 DST=127.0.0.1 LEN=60 TOS=0x00 PREC=0x00 TTL=64 ID=58288 DF PR SPT=53981 DPT=80 WINDOW=32767 RES=0x00 SYN URGP=0
```

The above log will also appear in `/var/log/messages`, `/var/log/syslog`, and `/var/log/kern.log`. This behavior can be modified by editing `/etc/syslog.conf` appropriately or by installing and configuring `ulogd` and using the `ULOG` target instead of `LOG`. The `ulogd` daemon is a userspace server that listens for logging instructions from the kernel – specifically for firewalls – and can log to any file you like, or even to a PostgreSQL or MySQL database. Making sense of your firewall logs can be simplified by using a log analysing tool such as `logwatch`, `fwanalog`, `fwlogwatch`, or `lire`.

Other tools

There are many tools available to help you construct a complete firewall without intimate knowledge of `iptables`. A command-line tool with plain-text configuration files, for example, is [Shorewall](#); a powerful solution to help you configure an advanced firewall for any network.

Further reading

- The [Ubuntu Firewall](#) wiki page contains information on the development of `ufw`
- Also, the `ufw(8)` manual page contains some very useful information: `man ufw`
- See the [packet-filtering how-to](#) for more information on using `iptables`
- The [nat how-to](#) contains further details on masquerading
- The [iptables how-to](#) in the Ubuntu wiki is a great resource

Ask a question on Discourse

Ask a question on Matrix

Open a GitHub issue for this page

[Edit this page on GitHub](#)[View page source](#)

Copyright © 2026

Last updated on Jun 26, 2026